

# Backward Pass for 3DGS Training

first version: L1 loss, fixed number of gaussians, one color per gaussian (SH degree 0)  
October 2, 2026

## The idea

Training repeats three steps. The *forward* pass renders the gaussians into an image. The *loss*  $L$  measures how far that image is from the real photo. The *backward* pass, this document, works out how much each gaussian’s numbers should change to shrink the loss. Then every number takes a small step:

$$x \leftarrow x - \text{lr} \cdot \bar{x}$$

where  $\text{lr}$  is the *learning rate*, a small number like 0.01 that sets how big each step is.

$\bar{x}$  (“ $x$  bar”) is the gradient  $\partial L / \partial x$ : how much the loss changes when  $x$  is nudged. It always has the same shape as  $x$ . Every formula below is the chain rule applied one step at a time, like a relay. Each section takes the gradient the previous one hands it and passes a new one along, from the pixel colors back to the five numbers we train.

| § | gives the gradient of                         | starts from        | runs in                      |
|---|-----------------------------------------------|--------------------|------------------------------|
| 1 | color $c_i$ , coverage $\alpha_i$             | $\bar{C}$          | raster                       |
| 2 | opacity $o_i$ , falloff $G_i$                 | $\bar{\alpha}_i$   | raster, last step in project |
| 3 | screen center $m$ , screen shape $\Sigma'$    | $\bar{G}_i$        | raster, last step in project |
| 4 | 3D shape $\Sigma$ , camera-space position $t$ | $\bar{\Sigma}'$    | project                      |
| 5 | position $\mu$                                | $\bar{m}, \bar{t}$ | project                      |
| 6 | rotation $q$ , size $\ell$                    | $\bar{\Sigma}$     | project                      |
| 7 | color coefficient $k$                         | $\bar{c}$          | project                      |

**The forward pass** first per gaussian, then per pixel  $p$ . The backward undoes exactly these steps.

$$\begin{aligned}
 t &= W\mu + \tau & \Sigma &= R \text{diag}(s^2)R^\top, \quad s = e^\ell, \quad R = R(q/|q|) \\
 m &= (f_x t_x / t_z + c_x, f_y t_y / t_z + c_y) & \Sigma' &= JW\Sigma W^\top J^\top + 0.3I, \quad Q = \Sigma'^{-1} \\
 G_i &= \exp(-\tfrac{1}{2}d^\top Qd), \quad d = p - m & \alpha_i &= \min(0.99, \rho_i G_i), \quad \rho_i = \sigma(o_i) \\
 c_i &= \max(0, C_0 k_i + 0.5) & C &= \sum_{i=1}^N c_i \alpha_i T_i + T_{N+1} b, \quad T_i = \prod_{j<i} (1 - \alpha_j)
 \end{aligned}$$

| symbol     | meaning                                                | symbol            | meaning                                       |
|------------|--------------------------------------------------------|-------------------|-----------------------------------------------|
| $\mu$      | 3D position in the world                               | $m$               | center on screen, in pixels                   |
| $q$        | rotation (quaternion)                                  | $\Sigma, \Sigma'$ | 3D shape, its 2D shape on screen              |
| $\ell, s$  | size per axis, $\ell = \log s$ keeps $s > 0$           | $Q$               | inverse of $\Sigma'$                          |
| $o, \rho$  | opacity, raw and squashed into 0..1                    | $G_i$             | strength of gaussian $i$ at $p$ (1 at center) |
| $k, c$     | color, stored coefficient and actual RGB               | $\alpha_i$        | how much of pixel $p$ it covers               |
| $t$        | position seen from the camera ( $t_z = \text{depth}$ ) | $T_i$             | light still getting through to $i$            |
| $W, \tau$  | camera rotation and translation                        | $C, C^*$          | our pixel, the photo’s pixel                  |
| $f_x, c_x$ | focal length, image center (pixels)                    | $b$               | background color                              |

**Two kinds of values** *Per pixel* values like  $\alpha_i$  and  $G_i$  are different at every pixel, so use them right away and never store them. *Per gaussian* values like  $\bar{c}_i$  get added into by many pixels at once, so use an *atomic add* (a GPU add that doesn’t lose updates when threads collide).

**Small rules** For symmetric matrices ( $Q, \Sigma', \Sigma$ ), store only the unique entries and copy them as is, with no doubling. Indices start at 0, so  $D_{02}$  is row 0, column 2. If the code stores plain size  $s$  instead of  $\ell$ , use  $\bar{s} = \bar{\ell}/s$ . Plain opacity  $\rho$  uses  $\bar{\rho}$  (§2), and plain RGB uses  $\bar{c}$  (§1).

## 1. Blending $\rightarrow$ gradients of color $c_i$ and coverage $\alpha_i$

*In words:* the forward stacks gaussians front to back like sheets of tinted glass. To undo it, walk the same list back to front.  $S_i$  is the color of everything behind gaussian  $i$ , background included, and  $T_i$  is recovered one step at a time.

$$\begin{aligned}
S_N &= b, & S_{i-1} &= \alpha_i c_i + (1 - \alpha_i) S_i, & T_i &= T_{i+1} / (1 - \alpha_i), & T_{N+1} &= T_{\text{final}} \\
\bar{c}_i &= \alpha_i T_i \bar{C} & & \text{per gaussian} \\
\bar{\alpha}_i &= T_i (c_i - S_i) \cdot \bar{C} & & \text{per pixel} \\
\bar{C} &= \text{sign}(C - C^*) / (3HW) & & \text{L1 loss: +1 if too bright, -1 if too dark}
\end{aligned}$$

Making  $i$  more opaque swaps some of what's behind it for its own color, weighted by how visible  $i$  is. The backward must visit exactly the gaussians the forward used. The forward skips faint ones (under  $1/255$ ) and stops once a pixel is nearly opaque, so it saves two numbers per pixel:  $T_{\text{final}}$ , and  $N$ , how far down the list it got.

## 2. Opacity $\rightarrow$ gradient of opacity $o_i$

*In words:* coverage is opacity times falloff, so its gradient splits between the two. The opacity part adds up over every pixel, and the squashing function's slope  $\rho(1 - \rho)$  is applied once at the end.

$$\begin{aligned}
\bar{G}_i &= \rho_i \bar{\alpha}_i & \text{per pixel} \\
\bar{\rho}_i &= \sum_p G_i \bar{\alpha}_i & \text{per gaussian} \\
\bar{o}_i &= \rho_i (1 - \rho_i) \bar{\rho}_i & \text{once per gaussian, after all pixels}
\end{aligned}$$

If  $\alpha$  hit the 0.99 cap, pass the gradient through anyway, like the original 3DGS code. The “skip if under  $1/255$ ” test must be written exactly like the forward's, or the two walk different lists.

## 3. Screen gaussian $\rightarrow$ gradients of center $m$ and shape $\Sigma'$

*In words:*  $G$  is a bell curve on screen, centered at  $m$  with shape  $Q$ . Moving the center or reshaping the bell changes  $G$  at every pixel, and those changes add up per gaussian. The last line converts the gradient for  $Q$  into one for  $\Sigma'$ , using the rule for differentiating a matrix inverse.

With  $w = G_i \bar{G}_i$  per pixel:

$$\begin{aligned}
\bar{m} &= \sum_p w Q d & \text{per gaussian} \\
\bar{Q} &= \sum_p -\frac{1}{2} w d d^\top & \text{per gaussian, 3 stored values} \\
\bar{\Sigma}' &= -Q \bar{Q} Q & \text{once per gaussian, from } dQ = -Q d\Sigma' Q
\end{aligned}$$

$d$  and  $\bar{m}$  are in pixels, so the forward has to compute  $m$  in pixels too.

## 4. Projection $\rightarrow$ gradients of 3D shape $\Sigma$ and position $t$

*In words:* this is the step that flattens the 3D shape onto the screen.  $W$  turns it to face the camera, and  $J$  is how perspective locally stretches things (farther away looks smaller).  $J$  depends on where the gaussian sits, so  $t$  gets a gradient here too.

$$J = \begin{bmatrix} f_x/t_z & 0 & -f_x t'_x/t_z^2 \\ 0 & f_y/t_z & -f_y t'_y/t_z^2 \end{bmatrix}, \quad t'_x = t_z \text{clamp}\left(\frac{t_x}{t_z}, \pm 1.3 \tan \frac{\text{fov}_x}{2}\right), \quad M = JW$$

With  $V = \bar{\Sigma}'$ , and  $k_x = 1$  if  $t_x$  is inside the clamp, else 0 (same for  $y$ ):

$$\begin{aligned} \bar{\Sigma} &= M^\top V M \\ D = \bar{J} &= 2 V J (W \Sigma W^\top) \\ \bar{t}_x &= -\frac{f_x}{t_z^2} D_{02} k_x \\ \bar{t}_y &= -\frac{f_y}{t_z^2} D_{12} k_y \\ \bar{t}_z &= -\frac{f_x}{t_z^2} D_{00} - \frac{f_y}{t_z^2} D_{11} + (1 + k_x) \frac{f_x t'_x}{t_z^3} D_{02} + (1 + k_y) \frac{f_y t'_y}{t_z^3} D_{12} \end{aligned}$$

The clamp keeps  $J$  from blowing up for gaussians far off to the side, and  $k_x$  switches that piece of the gradient off while it's active. The 2 in  $D$  is there because  $M$  appears twice in  $M \Sigma M^\top$ .

## 5. Screen center $\rightarrow$ gradient of position $\mu$

*In words:* where the gaussian lands on screen depends on its position. Add that to  $\bar{t}$  from §4, then turn it back from camera view to world view. Undoing a rotation is its transpose.

$$\begin{aligned} \bar{t}_x &+= \frac{f_x}{t_z} \bar{m}_x && \text{added to §4's } \bar{t} \\ \bar{t}_y &+= \frac{f_y}{t_z} \bar{m}_y \\ \bar{t}_z &+= -\frac{f_x t_x \bar{m}_x + f_y t_y \bar{m}_y}{t_z^2} \\ \bar{\mu} &= W^\top \bar{t} \end{aligned}$$

This uses the real  $t_x, t_y$ , not the clamped ones from §4.

## 6. Shape $\rightarrow$ gradients of rotation $q$ and size $\ell$

*In words:* the 3D shape is built from a rotation and a size per axis. Each axis's size only cares about the gradient along that axis:  $R^\top U R$  is the gradient seen from the gaussian's own axes. The entries of  $R$  are simple products of  $w, x, y, z$ , and the four  $\hat{q}$  lines are their derivatives collected. The last line accounts for  $q$  being normalized: stretching  $q$  doesn't change the rotation, so that direction is removed.

With  $U = \bar{\Sigma}$ ,  $\hat{q} = q/|q| = (w, x, y, z)$ , and  $\Gamma = \bar{R} = 2 U R \text{diag}(s^2)$ :

$$\begin{aligned} \bar{\ell}_k &= 2s_k^2 (R^\top U R)_{kk} \\ \bar{\tilde{q}}_w &= 2[z(\Gamma_{10} - \Gamma_{01}) + y(\Gamma_{02} - \Gamma_{20}) + x(\Gamma_{21} - \Gamma_{12})] \\ \bar{\tilde{q}}_x &= 2[y(\Gamma_{01} + \Gamma_{10}) + z(\Gamma_{02} + \Gamma_{20}) + w(\Gamma_{21} - \Gamma_{12})] - 4x(\Gamma_{11} + \Gamma_{22}) \\ \bar{\tilde{q}}_y &= 2[x(\Gamma_{01} + \Gamma_{10}) + w(\Gamma_{02} - \Gamma_{20}) + z(\Gamma_{12} + \Gamma_{21})] - 4y(\Gamma_{00} + \Gamma_{22}) \\ \bar{\tilde{q}}_z &= 2[w(\Gamma_{10} - \Gamma_{01}) + x(\Gamma_{02} + \Gamma_{20}) + y(\Gamma_{12} + \Gamma_{21})] - 4z(\Gamma_{00} + \Gamma_{11}) \\ \bar{q} &= (\bar{\tilde{q}} - \hat{q}(\hat{q} \cdot \bar{\tilde{q}}))/|q| \end{aligned}$$

$w$  is the real part. If the code stores the quaternion as  $(x, y, z, w)$ , match the letters, not the positions.

## 7. Color $\rightarrow$ gradient of color coefficient $k_i$

*In words:* with SH degree 0 the color is a constant times  $k$  plus 0.5, so its gradient is that constant times  $\bar{c}$ . Where the color was clamped to 0, nudging  $k$  changes nothing, so the gradient there is 0.

$$\bar{k}_i = C_0 \bar{c}_i, \quad 0 \text{ on channels where } C_0 k_i + 0.5 < 0.$$

Color that changes with viewing angle (SH degrees 1 to 3) comes later.

## 8. The two GPU programs

The raster backward runs first, one thread per pixel, and does the per-pixel parts (§1 to §3). The project backward runs after it, one thread per gaussian, and does the rest.

---

### Algorithm 1 Raster backward, one thread per pixel

---

```

1:  $T \leftarrow T_{\text{final}}, S \leftarrow b$ 
2: for  $i$  in the pixel's list, back to front from where the forward stopped do
3:   recompute  $d, G_i, \alpha_i$ ; skip if  $\alpha_i < 1/255$ 
4:    $T \leftarrow T/(1 - \alpha_i)$ 
5:    $\bar{c}_i += \alpha_i T \bar{C}$  ▷ atomic add
6:    $\bar{\alpha} \leftarrow T(c_i - S) \cdot \bar{C}, \quad S \leftarrow \alpha_i c_i + (1 - \alpha_i) S$ 
7:    $\bar{\rho}_i += G_i \bar{\alpha}$  ▷ atomic add
8:    $\bar{G} \leftarrow \rho_i \bar{\alpha}, \quad w \leftarrow G_i \bar{G}$ 
9:    $\bar{m}_i += w Q_i d, \quad \bar{Q}_i += -\frac{1}{2} w d d^\top$  ▷ atomic add
10: end for
```

---

▷ *atomic add*: thousands of pixel threads add into the same gaussian's sum at the same moment. A plain += is read, add, write, so two threads can read the same old value and one addition gets lost. An atomic add does all three as one step, so nothing is lost.

---

### Algorithm 2 Project backward, one thread per gaussian

---

```

1:  $\bar{o} \leftarrow \rho(1 - \rho) \bar{\rho}$  ▷ §2
2:  $\bar{\Sigma}' \leftarrow -Q \bar{Q} Q$  ▷ §3
3:  $\bar{\Sigma}, \bar{t}$  ▷ §4
4:  $\bar{t} += \dots, \quad \bar{\mu} \leftarrow W^\top \bar{t}$  ▷ §5
5:  $\bar{\ell}, \bar{q}$  ▷ §6
6:  $\bar{k}$  ▷ §7
```

---

| buffer                 | one entry per | holds                                                                                  |
|------------------------|---------------|----------------------------------------------------------------------------------------|
| saved by the forward   | pixel         | $T_{\text{final}}, N$                                                                  |
| raster backward's sums | gaussian      | $\bar{c}$ (3), $\bar{\rho}$ (1), $\bar{m}$ (2), $\bar{Q}$ (3), filled with atomic adds |
| final gradients        | gaussian      | $\bar{\mu}$ (3), $\bar{q}$ (4), $\bar{\ell}$ (3), $\bar{o}$ (1), $\bar{k}$ (3)         |

---